

Assignment 4

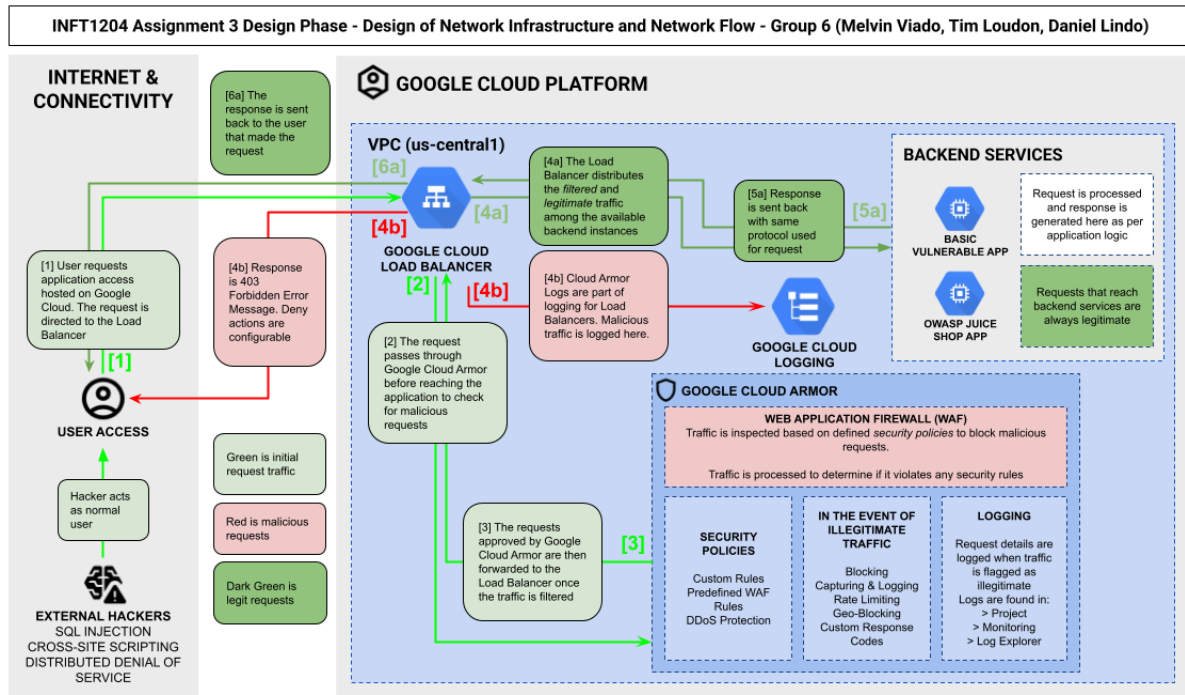
Timothy Loudon, Melvin Viado, Daniel Lindo

Vigna Thanikaslam

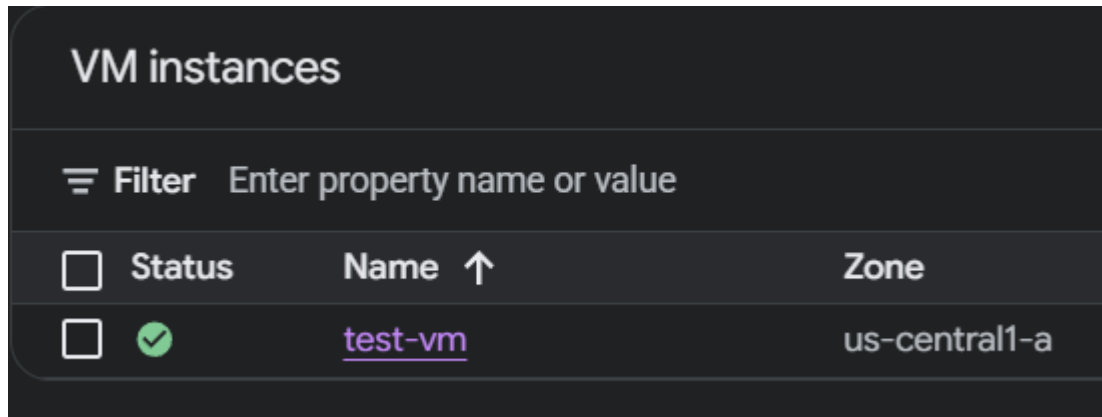
March 19th, 2026

WAF Diagram

Numbers indicate the order of traffic. Case “a” indicates legitimate traffic, case “b” indicates illegitimate traffic.



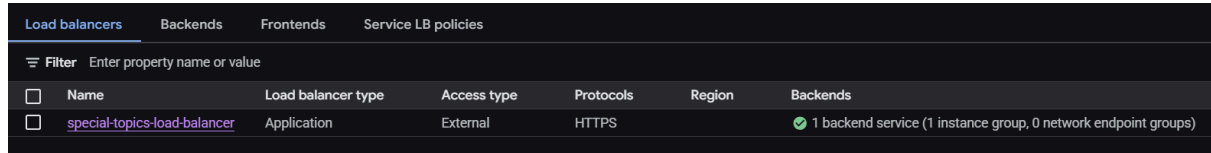
Virtual Machine running OWASP Juice Shop



VM instances			
Filter Enter property name or value			
☐	Status	Name ↑	Zone
☐	✓	test-vm	us-central1-a

Virtual Machine is also a part of an Unmanaged Instance Group (UMIG) as Load Balancers need to point at an Instance Group and not an individual VM instance. VM is a Debian instance running OWASP juice shop as a docker image.

Application Load Balancer



Load balancers Backends Frontends Service LB policies						
Filter Enter property name or value						
☐	Name	Load balancer type	Access type	Protocols	Region	Backends
☐	special-topics-load-balancer	Application	External	HTTPS		✓ 1 backend service (1 instance group, 0 network endpoint groups)

Load Balancer was created with a self-signed SSL certificate as it is impossible to create a google managed one without a domain name. When visiting the website users will get a warning that the connection is not secure because of this.

The Load Balancer is running a backend service listening in on the UMIG on port 3000 (the port designated to run OWASP Juice Shop). It also has a health check attached that is required for Load Balancers to function.

special-topics-backend-service

General properties

Load balancer type	Global external application load balancer (EXTERNAL_MANAGED)
Endpoint protocol	HTTP
In use by	special-topics-load-balancer
Named port	juiceshop
Timeout	30 seconds
IP address selection policy ⓘ	Only IPv4
Health check	juiceshop-health-check
Cloud CDN	Enabled View details
Logging	Enabled (sample rate: 1) All optional fields excluded
Backend security policy	special-topics-waf

⌵ Show advanced

Backends ⓘ

Name ↑	Type	IP stack type	Scope	Healthy	Auto-scaling	Balancing mode	Traffic duration	Selected ports ⓘ
special-topics-instance-group	Instance group	IPv4	us-central1-a	🟢 1 of 1	No configuration	Max backend utilisation: 80%	Default (short)	3000

Cloud Armor Security Policy

Cloud Armor policies ➕ Create policy 🔄 Refresh					
≡ Filter Enter property name or value					
☐	Name ↑	Type	Scope	Rules	Targets ②
☐	default-policy	Backend security policy	global	1	0
☐	special-topics-waf	Backend security policy	global	8	1

special-topics-waf

Type Backend security policy

Description

Scope global

Tags — ✎

✓ Show advanced configurations

Contains
8 rules

Applies to
1 target

[Rules](#) [Targets](#) [Logs](#)

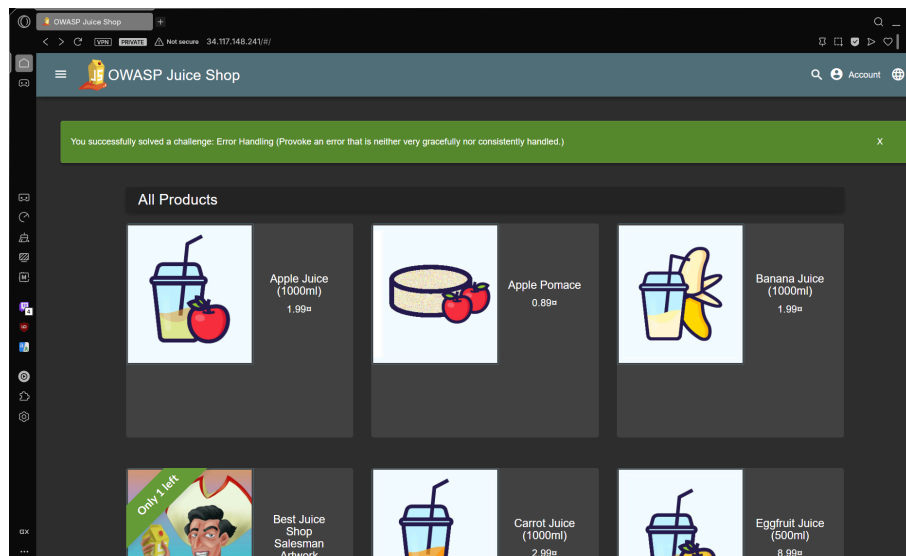
Rules are evaluated by priority: Lower numbers are evaluated first. [Learn more](#) 🔗

Add rule Delete More ▾

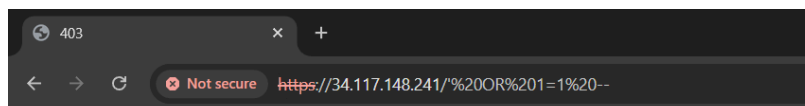
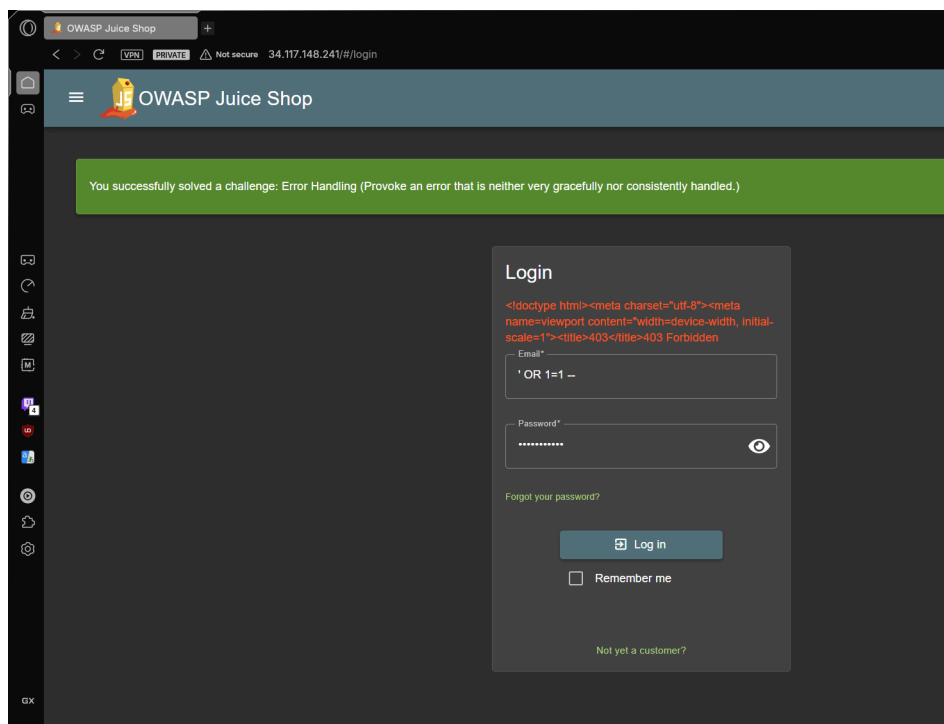
≡ Filter Enter property name or value					
☐	Action	Type	Match	Description	Priority ↑
☐	Deny (403)		evaluatePreconfiguredWaf('sqli-v33-stable')	Block SQL injection	1,000
☐	Deny (403)		evaluatePreconfiguredWaf('methodenforcement-v33-stable')	Block Method Enforcement	1,010
☐	Deny (403)		evaluatePreconfiguredWaf('xss-v33-stable')	Block Cross-Site Scripting	1,020
☐	Deny (403)		evaluatePreconfiguredWaf('scannerdetection-v33-stable')	Block Scanner Detection	1,030
☐	Deny (403)		evaluatePreconfiguredWaf('cve-canary')	Block CVE	1,040
☐	Deny (403)		evaluatePreconfiguredWaf('rce-v33-stable')	Block RCE	1,050
☐	Deny (403)	IP addresses/ranges	87.102.0.0/24	Blocking this guy in particular	1,060
☐	Allow	IP addresses/ranges	* (All IP addresses)	Default rule, higher priority overrides it	2,147,483,647

These are the rules implemented on the backend security policy which make up the Web Application Firewall. The OWASP rules already have pre-configurations from Google Cloud Armor and we simply need to turn it on by adding them into the security policy one by one. Each rule also has a different priority for easier log parsing. There is also a rule that blocks an IP address range to demonstrate how we can block various IP addresses with these rules.

WAF in Action



Example of SQL Injection and it being blocked by Cloud Armor



403 Forbidden

SQL Injection Attack being logged by Cloud Armor

The screenshot displays the Cloud Armor Logs Explorer interface. At the top, there's a 'Logs Explorer' header. Below it, a search bar labeled 'Search all fields' is present. A row of filters includes 'All resources', 'All log names', 'All severities', 'Correlate by', and a '+1 filter' button. The main log entry is numbered '1' and shows 'httpRequest.status=403'. Below the log entry, there are links for 'Sample queries' and 'Query language guide'. A section labeled '1 result' contains a table with columns 'Severity', 'Time', and 'Summary'. The table shows a log entry with a yellow warning icon, timestamp '2026-03-19 12:33:01.225', and a summary of the request details. Below the table, there are buttons for 'Investigate log', 'Copy', 'Expand nested fields', and 'Hide log summary'. The log details are expanded, showing a JSON object with fields like 'httpRequest', 'jsonPayload', 'cacheDecision', 'enforcedSecurityPolicy', 'preconfiguredExprIds', 'remoteIp', and 'securityPolicyRequestData'. The 'enforcedSecurityPolicy' field is highlighted, showing 'configuredAction: "DENY"', 'matchedFieldName: "id"', 'matchedFieldType: "ARG_VALUES"', 'matchedFieldValue: "'=''", 'matchedLength: 3', 'matchedOffset: 8', 'name: "special-topics-waf"', and 'outcome: "DENY"'. The 'preconfiguredExprIds' field shows 'priority: 1000'.

```
{
  httpRequest: {10}
  insertId: "1b7q8lkf13lg06"
  jsonPayload: {
    @type: "type.googleapis.com/google.cloud.loadbalancing.type.LoadBalancerLogEntry"
    backendTargetProjectNumber: "projects/544670548777"
    cacheDecision: [2]
    cacheId: "YYZ"
    cachePolicySource: "CACHE_POLICY_SOURCE_BACKEND"
    enforcedSecurityPolicy: {
      configuredAction: "DENY"
      matchedFieldLength: 63
      matchedFieldName: "id"
      matchedFieldType: "ARG_VALUES"
      matchedFieldValue: "'=''"
      matchedLength: 3
      matchedOffset: 8
      name: "special-topics-waf"
      outcome: "DENY"
    }
    preconfiguredExprIds: [1]
    priority: 1000
  }
  remoteIp: "65.94.104.220"
  securityPolicyRequestData: {3}
  statusDetails: "denied_by_security_policy"
}
```

Here you can see the SQL Injection attack attempt logged by Cloud Armor. It has details including the attacker IP address, the name of the security policy that was triggered, and the priority number which tells us what rule was triggered (in this case SQL Injection).

Other Things to Note

The Application Load Balancer also required a special firewall rule to allow external internet traffic going into it. If this rule doesn't exist, users would be unable to connect to the Load Balancer

☐	special-topics-firewall-rule	Ingress	Apply to all	IP ranges:	tcp:3000	Allow
--------------------------	--	---------	--------------	------------	----------	-------